

DAY - 31 : Master JavaScript Prototypes and Object Patterns - Confused to Confident! 🤖

JS JavaScript ES6+ JS Prototype Note Status Active Type Educational

Created by: Neeraj | [LinkedIn: neeraj-kumar1904](#) 📁 | [X: @_19_neeraj](#) 🐦 | [GitHub: Neeraj05042001](#) 🧑

Introduction

JavaScript Prototypes are one of the most powerful yet confusing concepts in JavaScript. They form the backbone of JavaScript's object-oriented programming model and inheritance system. Understanding prototypes is crucial for becoming a JavaScript master! 🚀

What is a Prototype?

A prototype is an object that serves as a template for other objects. Every JavaScript object has a prototype, and it can inherit properties and methods from its prototype.

OR

- Prototype is the internal property of an object which indicates from which structure the particular object has been created

JavaScript Object Literals Pattern

What are Object Literals?

Object literals are the most common way to create objects in JavaScript using curly braces `{}` .

```
// Object Literal Pattern
const person = {
  name: "John",
  age: 25,
  greet: function() {
    return `Hello, I'm ${this.name}`;
  }
};
```

```
console.log(person.name); // "John"
console.log(person.greet()); // "Hello, I'm John"
```

✅ Advantages:

- 🚀 Simple and quick to create
- 📖 Easy to read and understand
- 💡 Perfect for single-use objects

❌ Disadvantages:

- 🔄 No reusability
- 📦 Memory inefficient for multiple similar objects

▶ 💡 Knowledge Check: Object Literals

🎯 Ways To Extract Values From Objects

1. 🔗 Dot Notation

```
const user = { name: "Alice", age: 30 };
console.log(user.name); // "Alice"
```

2. 📁 Bracket Notation

```
const user = { name: "Alice", age: 30 };
console.log(user["name"]); // "Alice"
console.log(user["age"]); // 30
```

3. 📦 Destructuring Assignment

```
const user = { name: "Alice", age: 30, city: "New York" };

// Object Destructuring
const { name, age } = user;
console.log(name); // "Alice"
console.log(age); // 30

// With Renaming
const { name: userName, age: userAge } = user;
console.log(userName); // "Alice"
```

4. 🔍 Object.keys(), Object.values(), Object.entries()

```
const user = { name: "Alice", age: 30 };

console.log(Object.keys(user));    // ["name", "age"]
console.log(Object.values(user));  // ["Alice", 30]
console.log(Object.entries(user)); // [["name", "Alice"], ["age", 30]]
```

► 💡 Knowledge Check: Object Value Extraction

🔒 JavaScript Object and The const Keyword

📌 Important Concept:

When you declare an object with `const`, you cannot reassign the variable, but you **CAN** modify the object's properties!

```
const person = { name: "John", age: 25 };

// ✅ This works - modifying properties
person.name = "Jane";
person.age = 26;
console.log(person); // { name: "Jane", age: 26 }

// ❌ This throws an error - reassigning the variable
// person = { name: "Bob", age: 30 }; // TypeError!
```

🔒 To make objects truly immutable:

```
const person = Object.freeze({ name: "John", age: 25 });
// person.name = "Jane"; // This won't work in strict mode
```

► 💡 Knowledge Check: const with Objects

⚖️ JavaScript Functions vs Methods

🔧 Functions

Functions are independent blocks of code that can be called anywhere.

```
// Function Declaration
function greet(name) {
  return `Hello, ${name}!`;
}

// Function Expression
const greet = function(name) {
  return `Hello, ${name}!`;
};

// Arrow Function
const greet = (name) => `Hello, ${name}!`;
```

Methods

Methods are functions that belong to objects.

```
const person = {
  name: "John",
  // Method
  greet: function() {
    return `Hello, I'm ${this.name}`;
  },
  // ES6 Method Shorthand
  sayGoodbye() {
    return `Goodbye from ${this.name}`;
  }
};

console.log(person.greet()); // "Hello, I'm John"
```

Key Differences:

Functions	Methods
Independent	Belong to objects
<code>this</code> is undefined or global	<code>this</code> refers to the object
Can be called directly	Called on objects

►  Knowledge Check: Functions vs Methods

The Constructor Function Pattern

What is a Constructor Function?

A constructor function is a special function used to create and initialize objects. It's called with the `new` keyword.

```
// Constructor Function (PascalCase naming convention)
function Person(name, age) {
  this.name = name;
  this.age = age;
  this.greet = function() {
    return `Hello, I'm ${this.name}`;
  };
}

// Creating instances
const john = new Person("John", 25);
const jane = new Person("Jane", 30);

console.log(john.name); // "John"
console.log(jane.greet()); // "Hello, I'm Jane"
```

What happens when you use `new` ?

1.  Creates a new empty object
2.  Sets the prototype of the object
3.  Binds `this` to the new object
4.  Returns the new object (implicitly)

```
// Manual simulation of 'new'
function createPerson(name, age) {
  const obj = {}; // Step 1
  obj.__proto__ = Person.prototype; // Step 2
  Person.call(obj, name, age); // Step 3
  return obj; // Step 4
}
```

►  Knowledge Check: Constructor Functions

Composing Objects and References

Object Composition

Objects can contain other objects as properties, creating complex data structures.

```
const address = {
  street: "123 Main St",
  city: "New York",
  zipCode: "10001"
};

const person = {
  name: "John",
  age: 25,
  address: address // Object composition
};

console.log(person.address.city); // "New York"
```

References vs Copies

Objects are passed by reference, not by value!

```
const original = { name: "John", age: 25 };
const reference = original; // Reference, not copy!

reference.name = "Jane";
console.log(original.name); // "Jane" - original is modified!

// Creating a shallow copy
const shallowCopy = { ...original };
// or
const shallowCopy2 = Object.assign({}, original);

// Deep copy (for nested objects)
const deepCopy = JSON.parse(JSON.stringify(original));
```

►  Knowledge Check: Object References

Object Prototypes

What is a Prototype?

Every JavaScript object has a hidden property called `[[Prototype]]` (accessible via `__proto__` or `Object.getPrototypeOf()`).

```
const person = { name: "John" };

// Accessing prototype
console.log(person.__proto__); // Object.prototype
```

```
console.log(Object.getPrototypeOf(person)); // Object.prototype

// All objects inherit from Object.prototype
console.log(person.toString()); // "[object Object]"
```

Creating Objects with Prototypes

```
const animal = {
  speak: function() {
    return "Some sound";
  }
};

const dog = Object.create(animal);
dog.name = "Buddy";
dog.speak = function() {
  return "Woof!";
};

console.log(dog.speak()); // "Woof!"
```

►  Knowledge Check: Prototypes

Prototype Chain

What is the Prototype Chain?

The prototype chain is a mechanism that allows objects to inherit properties and methods from other objects.

```
// The chain: dog -> animal -> Object.prototype -> null
const animal = {
  eat: function() {
    return "Eating...";
  }
};

const dog = Object.create(animal);
dog.bark = function() {
  return "Woof!";
};

const myDog = Object.create(dog);
myDog.name = "Buddy";

// Prototype chain lookup
```

```
console.log(myDog.name); // "Buddy" (own property)
console.log(myDog.bark()); // "Woof!" (inherited from dog)
console.log(myDog.eat()); // "Eating..." (inherited from animal)
console.log(myDog.toString()); // "[object Object]" (inherited from Object.prototype)
```

Prototype Chain Visualization:

```
myDog
├─ name: "Buddy"
├─ __proto__: dog
│   └─ bark: function()
│       └─ __proto__: animal
│           └─ eat: function()
│               └─ __proto__: Object.prototype
│                   └─ toString: function()
│                       └─ __proto__: null
```

►  Knowledge Check: Prototype Chain

Constructor Function Visually

Visual Representation:

```
function Person(name) {
  this.name = name;
}

Person.prototype.greet = function() {
  return `Hello, I'm ${this.name}`;
};

const john = new Person("John");
const jane = new Person("Jane");
```

Memory Layout:

```
Person Constructor Function
├─ prototype: Person.prototype
│   └─ greet: function()
│       └─ constructor: Person
│
john instance
├─ name: "John"
└─ __proto__: Person.prototype
```



```
jane instance
├─ name: "Jane"
└─ __proto__: Person.prototype
```

► 💡 Knowledge Check: Constructor Visualization

🔧 The "prototype" Property

🎯 Understanding Function.prototype

Every function in JavaScript has a `prototype` property that's used as the prototype for instances created with `new`.

```
function Car(brand) {
  this.brand = brand;
}

// Adding methods to prototype
Car.prototype.start = function() {
  return `${this.brand} is starting...`;
};

Car.prototype.stop = function() {
  return `${this.brand} is stopping...`;
};

const toyota = new Car("Toyota");
const bmw = new Car("BMW");

console.log(toyota.start()); // "Toyota is starting..."
console.log(bmw.start());    // "BMW is starting..."

// Both instances share the same methods
console.log(toyota.start === bmw.start); // true
```

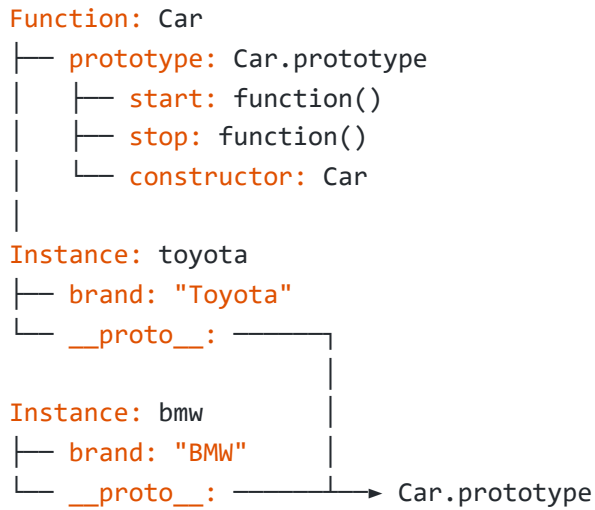
🔍 Important Points:

- 📦 `prototype` is only available on functions
- ⏰ It becomes the `__proto__` of instances created with `new`
- 💾 Methods on prototype are shared, saving memory

► 💡 Knowledge Check: prototype Property

The "prototype" Visually

Visual Representation:



Code Example:

```
function Animal(name) {
  this.name = name;
}

Animal.prototype.speak = function() {
  return `${this.name} makes a sound`;
};

const dog = new Animal("Dog");
const cat = new Animal("Cat");

// Visual proof
console.log(dog.__proto__ === Animal.prototype); // true
console.log(cat.__proto__ === Animal.prototype); // true
console.log(dog.speak === cat.speak); // true (same function reference)
```

Prototype To Save Memory

Memory Efficiency

Using prototypes prevents method duplication, saving memory when creating multiple instances.

```
// ❌ Memory Inefficient - Each instance has its own copy
function Person(name) {
```

```

    this.name = name;
    this.greet = function() {
        return `Hello, I'm ${this.name}`;
    };
}

// ✅ Memory Efficient - Shared method via prototype
function Person(name) {
    this.name = name;
}

Person.prototype.greet = function() {
    return `Hello, I'm ${this.name}`;
};

```

Memory Comparison:

```

// Creating 1000 instances
const people = [];
for (let i = 0; i < 1000; i++) {
    people.push(new Person(`Person${i}`));
}

// With prototype: 1000 objects + 1 shared method
// Without prototype: 1000 objects + 1000 individual methods

```

►  Knowledge Check: Memory Efficiency

Prototype Chain (Lookup) Visually

Property Lookup Process:

```

function Vehicle(type) {
    this.type = type;
}

Vehicle.prototype.start = function() {
    return "Starting vehicle...";
};

function Car(brand, type) {
    Vehicle.call(this, type);
    this.brand = brand;
}

Car.prototype = Object.create(Vehicle.prototype);

```

```
Car.prototype.constructor = Car;

Car.prototype.honk = function() {
  return "Beep beep!";
};

const myCar = new Car("Toyota", "Sedan");
```

Lookup Chain Visualization:

myCar.honk() lookup:

1. myCar (own properties) ✗
2. Car.prototype ✓ Found honk()

myCar.start() lookup:

1. myCar (own properties) ✗
2. Car.prototype ✗
3. Vehicle.prototype ✓ Found start()

myCar.toString() lookup:

1. myCar (own properties) ✗
2. Car.prototype ✗
3. Vehicle.prototype ✗
4. Object.prototype ✓ Found toString()

JavaScript Class Pattern

ES6 Classes

Classes provide a cleaner syntax for creating constructor functions and prototypes.

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  greet() {
    return `Hello, I'm ${this.name}`;
  }

  getAge() {
    return this.age;
  }

  static species() {

```

```
        return "Homo sapiens";
    }
}

const john = new Person("John", 25);
console.log(john.greet()); // "Hello, I'm John"
console.log(Person.species()); // "Homo sapiens"
```

Class vs Constructor Function:

```
// Class syntax
class Car {
    constructor(brand) {
        this.brand = brand;
    }

    start() {
        return `${this.brand} is starting...`;
    }
}

// Equivalent constructor function
function Car(brand) {
    this.brand = brand;
}

Car.prototype.start = function() {
    return `${this.brand} is starting...`;
};
```

►  Knowledge Check: Classes

JavaScript Class Inheritance

Extending Classes

```
class Animal {
    constructor(name) {
        this.name = name;
    }

    speak() {
        return `${this.name} makes a sound`;
    }
}
```

```

class Dog extends Animal {
  constructor(name, breed) {
    super(name); // Call parent constructor
    this.breed = breed;
  }

  speak() {
    return `${this.name} barks`;
  }

  wagTail() {
    return `${this.name} wags tail`;
  }
}

const myDog = new Dog("Buddy", "Golden Retriever");
console.log(myDog.speak()); // "Buddy barks"
console.log(myDog.wagTail()); // "Buddy wags tail"

```

Key Concepts:

- **extends** : Creates inheritance relationship
- **super()** : Calls parent constructor
- **Method overriding**: Child methods override parent methods

►  Knowledge Check: Class Inheritance

The Object.create() Pattern

Alternative to Constructor Functions

`Object.create()` creates a new object with a specified prototype.

```

const animal = {
  type: "Animal",
  speak: function() {
    return `${this.name} makes a sound`;
  }
};

// Create object with animal as prototype
const dog = Object.create(animal);
dog.name = "Buddy";
dog.speak = function() {
  return `${this.name} barks`;
};

```

```
console.log(dog.speak()); // "Buddy barks"  
console.log(dog.type); // "Animal" (inherited)
```

Object.create() vs new:

```
// Using Object.create()  
const dog1 = Object.create(animal);  
dog1.name = "Rex";  
  
// Using constructor function  
function Animal(name) {  
    this.name = name;  
}  
const dog2 = new Animal("Max");
```

Benefits:

-  More explicit prototype setting
-  No need for constructor functions
-  Flexible object creation

►  Knowledge Check: Object.create()

How To Get Prototype Of An Object

Methods to Access Prototype:

```
const person = { name: "John" };  
  
// Method 1: Object.getPrototypeOf() (Recommended)  
console.log(Object.getPrototypeOf(person)); // Object.prototype  
  
// Method 2: __proto__ property (Deprecated)  
console.log(person.__proto__); // Object.prototype  
  
// Method 3: constructor.prototype (for instances)  
function Car(brand) {  
    this.brand = brand;  
}  
const myCar = new Car("Toyota");  
console.log(myCar.constructor.prototype); // Car.prototype
```

Checking Prototype Chain:

```
function Animal() {}
function Dog() {}
Dog.prototype = Object.create(Animal.prototype);

const myDog = new Dog();

// Check if object is in prototype chain
console.log(Animal.prototype.isPrototypeOf(myDog)); // true
console.log(Dog.prototype.isPrototypeOf(myDog)); // true
```

► 💡 Knowledge Check: Getting Prototypes

⚙️ How To Set An Object Prototype

🎯 Methods to Set Prototype:

```
const animal = {
  speak: function() {
    return "Some sound";
  }
};

// Method 1: Object.setPrototypeOf() (Use with caution)
const dog = { name: "Buddy" };
Object.setPrototypeOf(dog, animal);
console.log(dog.speak()); // "Some sound"

// Method 2: Object.create() (Recommended)
const cat = Object.create(animal);
cat.name = "Whiskers";

// Method 3: Using constructor and prototype
function Bird(name) {
  this.name = name;
}
Bird.prototype = animal;
const parrot = new Bird("Polly");
```

⚠️ Important Notes:

- 🐢 `Object.setPrototypeOf()` is slow and should be avoided
- ⚖️ `Object.create()` is preferred for setting prototypes
- 🎯 Set prototypes at object creation time, not later

► 💡 Knowledge Check: Setting Prototypes

The Closing Notes

Key Takeaways:

1. **Prototypes** are the foundation of JavaScript inheritance
2. **Constructor functions** create objects with shared prototypes
3. **Classes** are syntactic sugar over prototypes
4. **Prototype chain** enables property inheritance
5. **Memory efficiency** is achieved through prototype methods

Best Practices:

-  Use `Object.getPrototypeOf()` instead of `__proto__`
-  Prefer `Object.create()` for setting prototypes
-  Use classes for cleaner syntax
-  Add methods to prototypes for memory efficiency
-  Avoid `Object.setPrototypeOf()` for performance

Common Interview Questions

- ▶  Q1: What's the difference between `__proto__` and `prototype` ?
- ▶  Q2: How does prototype chain work?
- ▶  Q3: What happens when you call a function with `new` ?
- ▶  Q4: How do you create an object without a prototype?
- ▶  Q5: What's the difference between classical inheritance and prototypal inheritance?

Practice Tasks

Task 1: Basic Prototype Understanding

```
// Create a constructor function 'Vehicle' with properties 'brand' and 'year'
// Add a method 'getInfo' to the prototype that returns brand and year
// Create two instances and test the method

// Your code here...
```

► 💡 Solution

Task 2: Prototype Chain

```
// Create a prototype chain: Animal -> Dog -> Labrador
// Each level should have its own method
// Test method inheritance

// Your code here...
```

► 💡 Solution

Task 3: Class Inheritance

```
// Convert the above prototype chain to use ES6 classes
// Include proper constructor chaining with super()

// Your code here...
```

► 💡 Solution

Task 4: Object.create() Pattern

```
// Create a base object 'shape' with area() method
// Create specific shapes (circle, rectangle) using Object.create()
// Each shape should have its own area calculation

// Your code here...
```

► 💡 Solution

Task 5: Prototype Method Sharing

```
// Create a Calculator constructor with basic operations
// Add methods to prototype for memory efficiency
// Test with multiple instances

// Your code here...
```

► 💡 Solution

Task 6: Prototype Chain Debugging

```
// Debug this code - find and fix the prototype chain issues
function Animal() {}
Animal.prototype.speak = function() { return "Sound"; };

function Dog() {}
Dog.prototype.speak = function() { return "Woof"; };

function Puppy() {}
Puppy.prototype.play = function() { return "Playing"; };

const myPuppy = new Puppy();
console.log(myPuppy.speak()); // Should work but doesn't
```

► 💡 Solution

Task 7: Mixed Patterns

```
// Create a library management system using:
// 1. Constructor function for Book
// 2. Class for Library
// 3. Object.create() for SpecialBook
// Implement proper inheritance and method sharing

// Your code here...
```

► 💡 Solution

Task 8: Prototype Performance

```
// Create two versions of a Person constructor:
// Version 1: Methods in constructor (inefficient)
// Version 2: Methods in prototype (efficient)
// Compare memory usage conceptually

// Your code here...
```

► 💡 Solution

Task 9: Advanced Prototype Manipulation

```
// Create a plugin system where base functionality can be extended
// Use prototype manipulation to add features dynamically

// Your code here...
```

► 💡 Solution

Task 10: Real-world Scenario

```
// Create a game character system with:  
// - Base Character class  
// - Warrior, Mage, Archer subclasses  
// - Equipment system using composition  
// - Skill system using prototype extension  
  
// Your code here...
```

► 💡 Solution

Final Assessment Quiz

Test Your Knowledge (10 Questions)

1. What is the main difference between `__proto__` and `prototype` ?

- A) They are the same thing
- B) `__proto__` is for objects, `prototype` is for functions
- C) `__proto__` is newer syntax
- D) `prototype` is deprecated

2. When you create an object with `new` , what happens first?

- A) The function is called
- B) A new empty object is created
- C) `this` is bound
- D) The prototype is set

3. How many `greet` methods exist in memory with this code?

```
function Person(name) { this.name = name; }  
Person.prototype.greet = function() { return "Hello"; };  
const p1 = new Person("John");  
const p2 = new Person("Jane");
```

- A) 0
- B) 1
- C) 2

- D) 3

4. What does `Object.create(null)` create?

- A) An empty object with `Object.prototype`
- B) An object with no prototype
- C) A null value
- D) An error

5. In class inheritance, what does `super()` do?

- A) Creates a new object
- B) Calls the parent constructor
- C) Sets the prototype
- D) Returns the parent class

6. Which is the most efficient way to add methods to objects?

- A) In the constructor function
- B) Using `Object.assign()`
- C) On the prototype
- D) As static methods

7. What happens in prototype chain lookup?

- A) Searches from parent to child
- B) Searches from child to parent
- C) Searches all objects simultaneously
- D) Searches randomly

8. Which method is recommended for getting an object's prototype?

- A) `obj.__proto__`
- B) `obj.prototype`
- C) `Object.getPrototypeOf(obj)`
- D) `obj.constructor.prototype`

9. What's the relationship between ES6 classes and prototypes?

- A) Classes replace prototypes
- B) Classes are syntactic sugar over prototypes
- C) Classes and prototypes are unrelated
- D) Prototypes are deprecated

10. Why should you avoid `Object.setPrototypeOf()` ?

- A) It's not supported
- B) It's deprecated
- C) It can cause performance issues
- D) It doesn't work with classes

►  Answer Key

Keep coding and stay curious! 🌟

Understanding prototypes is like understanding the DNA of JavaScript objects - once you get it, everything else makes sense! 💡

 ***Let's Connect & Level Up Together***

Follow me for daily JavaScript tips, insightful notes, and project-based learning.

 [X \(Twitter\)](#)

 [LinkedIn](#)

 [Telegram](#)

 [Threads](#)

 Dive into the full notes on GitHub → [40 Days JavaScript](#)

© 2025 • Crafted with ❤️ by Neeraj

 Next: Day - 32: Master JavaScript Modules: import, export, and Organize Like a Pro! 🎉 

 Happy JavaScript coding! 